

SỔ TAY GIAO TIẾP, BẢO MẬT VÀ QUAN SÁT HỆ THỐNG COMMERCE

Kiến trúc microservices .NET · Docker local · Hướng dẫn vận hành và tùy biến

Phạm vi	Identity, Gateway, Catalog, Cart, Ordering, Inventory, Payment, Shipping, Notification
Hạ tầng	PostgreSQL, Redis, RabbitMQ, OpenTelemetry Collector, Prometheus, Grafana, Jaeger
Cơ sở	Mã nguồn và cấu hình Docker hiện tại trong repository Commerce
Ngày phát hành	29/09/2026 · Phiên bản 1.0

Tài liệu vận hành dành cho developer, reviewer và người trực hệ thống local.

Cách dùng tài liệu

Tài liệu này mô tả đúng hiện trạng của repository tại thời điểm phát hành, sau đó tách riêng các khuyến nghị để nâng cấp production. Khi có khác biệt, mã nguồn và các tệp cấu hình được liệt kê ở cuối tài liệu là nguồn sự thật ưu tiên.

Đọc nhanh: Chương 1–4 để hiểu kiến trúc và checkout; Chương 5–7 để làm việc với bảo mật/RabbitMQ; Chương 8–11 để quan sát; Chương 12 để xử lý sự cố và hardening.

Quy ước: “Hiện tại” nghĩa là đã có trong repo. “Khuyến nghị production” không ngụ ý hệ thống hiện tại đã triển khai nội dung đó.

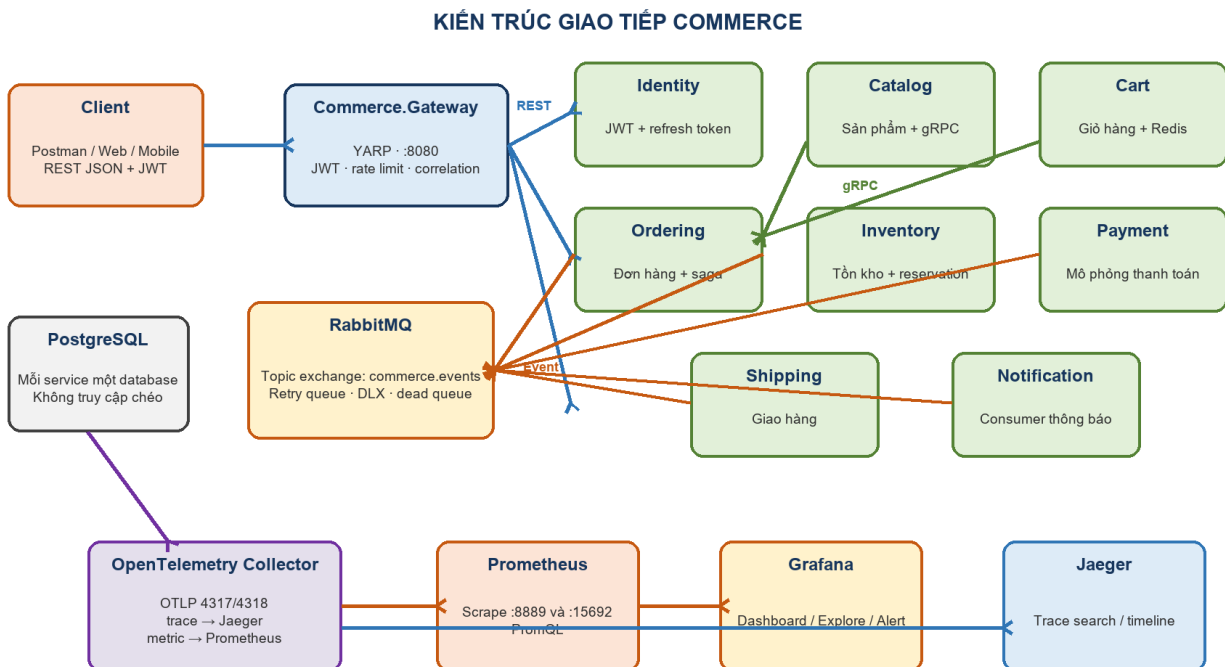
An toàn bí mật: Ví dụ chỉ dùng tên biến và giá trị giả. Không chép JWT, mật khẩu, refresh token hoặc connection string thật vào tài liệu, dashboard hay log.

Mục lục

- 1. Bức tranh tổng thể và nguyên tắc biên service
- 2. Bản đồ service, dữ liệu và cổng local
- 3. REST, gRPC và event: chọn đúng kiểu giao tiếp
- 4. Luồng checkout saga từ đầu đến cuối
- 5. Xác thực bằng ASP.NET Core Identity và JWT
- 6. Phân quyền, ownership và kiểm soát tại Gateway/service
- 7. RabbitMQ: cấu hình, độ tin cậy, retry, DLQ và tùy biến
- 8. OpenTelemetry Collector: thu nhận và định tuyến telemetry
- 9. Prometheus: scrape, PromQL, recording rule và alert rule
- 10. Grafana: dashboard, Explore, biến, cảnh báo và provisioning
- 11. Jaeger: tìm trace, đọc critical path và tùy biến
- 12. Runbook xử lý sự cố và hardening production
- Phụ lục A–D. Lệnh nhanh, ma trận routing, cấu hình và nguồn tham khảo

1. Bức tranh tổng thể và nguyên tắc biên service

Commerce là hệ thống microservices .NET chạy local bằng Docker Compose. Người dùng chỉ gọi REST qua Commerce.Gateway. Các service không đọc database của nhau; truy vấn đồng bộ nội bộ dùng gRPC, còn thay đổi nghiệp vụ dài và cần chống lỗi dùng RabbitMQ.



Hình 1 — Toàn cảnh các kênh giao tiếp và pipeline quan sát.

1.1 Bốn nguyên tắc cốt lõi

Nguyên tắc	Ý nghĩa thực tế
Một cửa công khai	Client đi qua Gateway/YARP tại <code>http://localhost:8080</code> . Service API và gRPC dùng mạng Docker nội bộ.
Sở hữu dữ liệu	Mỗi service có database PostgreSQL riêng; Cart sở hữu Redis. Không join hoặc ghi chéo database.
Đồng bộ có giới hạn	gRPC chỉ phục vụ truy vấn dữ liệu cần ngay, có deadline 3 giây; không biến gRPC thành giao dịch phân tán.
Bất đồng bộ có bằng chứng	Event được ghi Outbox cùng transaction nghiệp vụ, publish có confirm, consumer có Inbox chống xử lý lặp.

1.2 Điều gì không được làm

- Không cho Gateway chứa domain logic hoặc truy cập database của service.
- Không công khai gRPC, RabbitMQ hay PostgreSQL ra Internet chỉ để tiện debug.
- Không dùng một event mutable cho nhiều phiên bản consumer; thay đổi breaking phải phát hành routing key/contract v2.
- Không coi HTTP 202 của checkout là đơn hàng đã hoàn tất; đó chỉ là xác nhận saga đã được chấp nhận.
- Không lưu token, mật khẩu, PII hoặc connection string trong span attribute, metric label hay log message.

2. Bản đồ service, dữ liệu và cổng local

2.1 Trách nhiệm và kho dữ liệu

Thành phần	Trách nhiệm	Kho sở hữu
Commerce.Gateway	YARP routing, JWT validation, rate limit, correlation	Không có database
Identity.Api	Đăng ký, đăng nhập, refresh, logout, phát hành JWT	commerce_identity
Catalog.Api ×2	Sản phẩm; REST công khai đọc; gRPC nội bộ	commerce_catalog
Cart.Api	Giỏ hàng theo customer	Redis
Ordering.Api	Checkout, Order aggregate, saga, outbox/inbox	commerce_ordering
Inventory.Api	Tồn kho, reservation lease, nhận hàng	commerce_inventory
Payment.Api	Mô phỏng kết quả thanh toán	commerce_payment
Shipping.Api	Tạo và cập nhật shipment	commerce_shipping
Notification.Worker	Tiêu thụ event và lưu thông báo	commerce_notification

2.2 Cổng từ máy host

Thành phần	Cổng local	Mục đích
Gateway	8080	Public REST entry point
PostgreSQL	127.0.0.1:\${POSTGRES_PORT:-5432}	DB management local; bind loopback
Redis	127.0.0.1:\${REDIS_PORT:-6379}	Redis management local; hiện không có auth
RabbitMQ Management	15672	UI; AMQP 5672 chỉ ở mạng Docker
Jaeger	16686	Trace UI
Prometheus	9090	Query/Targets UI
Grafana	3000	Dashboard/Explore/Alert
OTel Collector	không publish ra host	OTLP 4317/4318 và exporter 8889 chỉ nội bộ

Lưu ý mạng: Tên host trong container là tên service Docker như postgres, redis, rabbitmq, otel-collector. Từ Navicat/Redis client trên Windows dùng 127.0.0.1 và cổng đã publish.

2.3 Khởi động và kiểm tra nhanh

```
Copy-Item .env.example .env
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
docker compose --env-file .env logs --tail 100 gateway identity-api ordering-api
```

Giữ `.env` ngoài Git. `.env.example` chỉ là mẫu local. Nếu thay cổng Redis bằng `REDIS\_PORT`, thêm biến đó vào `.env`; nếu bỏ trống, Compose dùng 6379.

3. REST, gRPC và event: chọn đúng kiểu giao tiếp

3.1 REST qua Gateway

REST phù hợp với request/response do người dùng khởi tạo. Gateway route theo prefix `/api/...`, chuyển tiếp header Authorization và correlation, đồng thời áp dụng token-bucket rate limit theo claim `sub` hoặc địa chỉ IP. Cấu hình hiện tại: 100 token, nạp 50 token mỗi 10 giây, queue tối đa 10; vượt ngưỡng trả 429.

Route	Đích	Ngữ nghĩa
/api/auth/*	Identity.Api	register, login, refresh, logout
/api/catalog/*	Catalog.Api (round-robin 2 instance)	đọc sản phẩm; Admin quản trị
/api/cart/*	Cart.Api	giỏ hàng của customer
/api/orders/*	Ordering.Api	checkout và đọc đơn
/api/inventory/*	Inventory.Api	Admin xem/nhập tồn
/api/payments/*	Payment.Api	Admin tra thanh toán
/api/shipping/*	Shipping.Api	tra shipment; Admin advance

3.2 gRPC nội bộ

Caller	RPC	Mục đích	Deadline
Cart → Catalog	GetProduct	Kiểm tra snapshot sản phẩm khi cập nhật giỏ	3 giây
Ordering → Cart	GetCartForCheckout	Lấy giỏ hiện tại khi checkout	3 giây
Ordering → Catalog	GetProducts	Lấy nhiều snapshot sản phẩm	3 giây

gRPC chạy HTTP/2 ở cổng container 8081. Deadline ngăn thread treo vô hạn, nhưng caller vẫn phải xử lý timeout như một lỗi có thể retry ở biên phù hợp. Không dùng gRPC để cập nhật đồng thời hai database.

3.3 RabbitMQ cho business event

RabbitMQ phù hợp khi publisher không cần đợi toàn bộ chuỗi nghiệp vụ hoàn thành. Routing key chứa capability, sự kiện và version, ví dụ `ordering.payment-requested.v1`. Consumer queue có tên riêng nên mỗi service nhận bản sao độc lập.

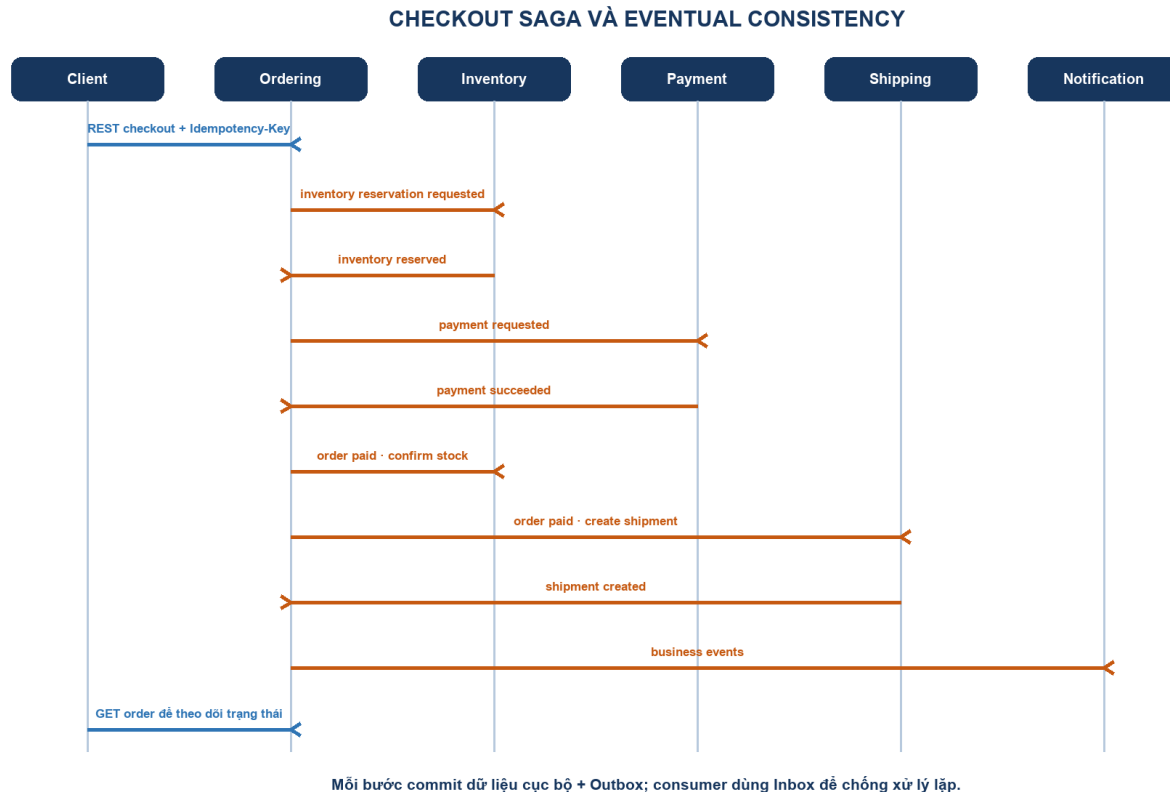
Câu hỏi	REST	gRPC	RabbitMQ
Cần kết quả ngay?	Có	Có	Không
Caller là client?	Có	Không	Không
Ghép lồng / fan-out?	Thấp	Thấp	Cao
Lỗi downstream?	Trả lỗi/timeout	Status/timeout	Retry + DLQ
Nhất quán	Trong request	Trong request	Eventual consistency

3.4 Correlation và trace propagation

Middleware đọc `X-Correlation-Id`; nếu thiếu hoặc không hợp lệ, hệ thống tạo GUID mới và trả header này trong response. Với HTTP, OpenTelemetry truyền W3C `traceparent`/`tracestate`. Với RabbitMQ, publisher ghi các header trace vào message; consumer tạo span mới nhưng giữ quan hệ trace để Jaeger hiển thị chuỗi end-to-end.

```
$headers = @{
    Authorization = "Bearer <access-token>"
    "X-Correlation-Id" = [guid]::NewGuid().ToString()
    "Idempotency-Key" = [guid]::NewGuid().ToString()
}
Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/orders/checkout `
-Headers $headers -ContentType "application/json" -Body "{}"
```

4. Luồng checkout saga từ đầu đến cuối



Hình 2 — Happy path của checkout saga; các mũi tên màu cam là event RabbitMQ.

4.1 Happy path

1. Client gửi checkout có Bearer token và `Idempotency-Key`.
2. Ordering xác định customer từ claim `sub`, lấy cart qua gRPC, lấy snapshot sản phẩm qua Catalog gRPC.
3. Ordering commit Order, SagaState và Outbox message trong một transaction; API trả 202 Accepted.
4. Outbox publisher gửi `ordering.inventory-reservation-requested.v1` và chỉ đánh dấu processed sau publisher confirm.
5. Inventory tạo reservation rồi phát `inventory.reserved.v1`; nếu thiếu hàng phát `inventory.reservation-failed.v1`.
6. Ordering nhận kết quả tồn kho. Nếu thành công, phát `ordering.payment-requested.v1`.
7. Payment phát `payment.succeeded.v1` hoặc `payment.failed.v1`.
8. Nếu thanh toán thành công, Ordering chuyển Order sang Paid và phát `ordering.order-paid.v1`; Inventory xác nhận stock, Shipping tạo shipment.
9. Notification tiêu thụ các event quan trọng. Client gọi GET order để thấy trạng thái cuối cùng.

4.2 Nhánh lỗi và bù trừ

- Reservation thất bại hoặc hết hạn: saga hủy đơn; không gửi payment request.
- Payment thất bại: Ordering phát inventory release request và order cancelled.
- Consumer lỗi tạm thời: message đi retry queue 5 giây rồi quay về queue chính.
- Lỗi quá số lần: message vào dead queue; operator phải điều tra nguyên nhân trước khi replay.
- Event phát trùng: Inbox ngăn mutation lặp theo cặp MessageId + Consumer.

4.3 Idempotency khác retry như thế nào

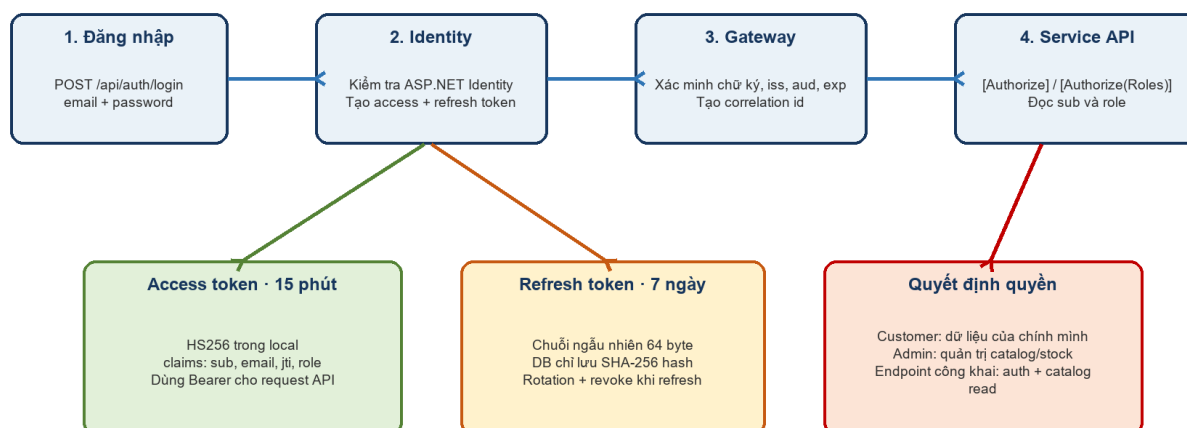
Retry đảm bảo thao tác có cơ hội chạy lại; idempotency đảm bảo chạy lại không tạo kết quả nghiệp vụ lặp. `Idempotency-Key` bảo vệ lệnh checkout từ client. MessageId/Inbox bảo vệ consumer. Outbox bảo vệ khoảng hở giữa commit database và publish broker. Ba cơ chế giải quyết ba điểm lỗi khác nhau và phải tồn tại đồng thời.

4.4 Cách theo dõi một checkout

1. Ghi lại `X-Correlation-Id`, order id và thời điểm gửi request.
2. Trong Jaeger, chọn service `Commerce.Gateway` hoặc `Ordering.Api`, lookback phù hợp, lọc operation/request path nếu cần.
3. Trong RabbitMQ UI, kiểm tra queue Ready/Unacked/Consumers và dead queue của consumer bị nghi ngờ.
4. Trong Prometheus/Grafana, đối chiếu request rate, latency và RabbitMQ backlog đúng khoảng thời gian.
5. Chỉ kiểm tra DB của service sở hữu dữ liệu; không sửa trạng thái trực tiếp để “chữa” saga.

5. Xác thực bằng ASP.NET Core Identity và JWT

LUỒNG XÁC THỰC VÀ PHÂN QUYỀN



Không gửi token vào log, metric, trace hoặc dashboard.

Hình 3 — Access token dùng cho API; refresh token chỉ dùng để xin access token mới.

5.1 Cách Identity tạo token

Identity.Api dùng ASP.NET Core Identity để quản lý user, password hash, lockout và role. Password policy local yêu cầu tối thiểu 12 ký tự, chữ hoa, chữ thường, chữ số và ký tự đặc biệt; email là duy nhất; lockout sau tối đa 5 lần thất bại.

Token	Thời hạn	Hình thức	Nội dung/lưu trữ	Cách dùng
Access token	15 phút	JWT HS256	sub, email, jti, role	Authorization: Bearer
Refresh token	7 ngày	64 byte ngẫu nhiên	DB chỉ lưu SHA-256 hash	POST /api/auth/refresh

Khi refresh, token cũ bị revoke và liên kết tới hash của token thay thế. Logout revoke refresh token. Access token đã phát không được thu hồi tức thì; thời hạn ngắn giới hạn cửa sổ rủi ro.

5.2 Secret key nằm ở đâu và chạy qua các lớp config

Lớp	Tên	Vai trò
`.env` local	JWT_SIGNING_KEY	Giá trị bí mật thật trên máy developer; không commit
docker-compose.yml	Jwt__SigningKey	Map environment variable vào ASP.NET Core configuration
.NET configuration	Jwt:SigningKey	Identity dùng để ký; Gateway và API dùng để xác minh
Mã nguồn	AddCommerceJwt / token service	Validate issuer, audience, lifetime, signature; Identity ký HS256

```

# .env – ví dụ, không dùng giá trị này ngoài local
JWT_ISSUER=commerce.identity
JWT_AUDIENCE=commerce.apis
JWT_SIGNING_KEY=<chuoi-ngau-nhien-dai-va-bi-mat>

# Docker Compose chuyển thành hierarchical configuration
Jwt__Issuer=${JWT_ISSUER}
Jwt__Audience=${JWT_AUDIENCE}
Jwt__SigningKey=${JWT_SIGNING_KEY}
  
```

Vì sao mọi API cần key? Với HS256, cùng một secret vừa ký vừa xác minh. Vì vậy hiện tại Identity, Gateway và các API đều nhận key. Đây là thuận tiện local nhưng mở rộng blast radius nếu một service bị lộ.

Khuyến nghị production: Dùng OIDC/OAuth 2.0 với RS256/ES256 và JWKS. Chỉ Identity/authorization server giữ private key; Gateway và service chỉ nhận public key. Đưa secret/private key vào secret manager và xoay khóa có `kid`.

5.3 Thử đăng nhập và gọi API

```
$loginBody = @{
    email    = "<customer-email-from-.env>"
    password = "<customer-password-from-.env>"
} | ConvertTo-Json

$tokens = Invoke-RestMethod -Method Post `
    -Uri http://localhost:8080/api/auth/login `
    -ContentType "application/json" -Body $loginBody

Invoke-RestMethod -Uri http://localhost:8080/api/cart `
    -Headers @{ Authorization = "Bearer $($tokens.accessToken)" }
```

Không paste access token vào issue, screenshot hoặc query URL. Nếu cần debug claim, dùng công cụ nội bộ/offline và che chữ ký/token trước khi chia sẻ.

5.4 Gateway và service cùng validate

Gateway validate sớm để chặn token sai và chuẩn hóa security boundary. Service tiếp tục validate `[Authorize]` vì mạng nội bộ không phải là trust boundary tuyệt đối. Cấu hình hiện tại tắt inbound claim mapping, dùng claim `role` làm role claim, và cho phép clock skew 30 giây.

6. Phân quyền, ownership và kiểm soát tại Gateway/service

6.1 Ma trận quyền hiện tại

Nhóm endpoint	Quyền	Cách enforce/ghi chú
Auth register/login/refresh	Anonymous	Identity xử lý credential/token
Catalog GET	Anonymous	Public read
Catalog create/update/activate/deactivate	Admin	Role claim
Cart GET/PUT/DELETE	Authenticated	Customer id lấy từ `sub`
Order checkout/read	Authenticated	Customer-scoped bằng `sub`
Inventory GET/receipts	Admin	Role claim
Payment GET by order	Admin	Role claim
Shipping GET	Authenticated	Hiện chưa kiểm tra owner trong action
Shipping advance	Admin	Role claim

Điểm cần review: Shipping GET hiện yêu cầu authenticated nhưng chưa chứng minh shipment thuộc customer đang gọi. Trước production nên thêm ownership check hoặc chỉ cho Admin, tùy use case.

6.2 Authentication khác authorization

- Authentication trả lời “ai đang gọi?” bằng chữ ký JWT và claim `sub`.
- Role authorization trả lời “người này có vai trò nào?” bằng claim `role`.
- Resource authorization trả lời “người này có sở hữu order/shipment này không?” bằng so sánh `sub` với dữ liệu thuộc service.
- Business authorization trả lời “trạng thái hiện tại có cho phép thao tác không?” trong domain/application layer.

6.3 Cách thêm policy tùy biến

Khi rule vượt quá role đơn giản, tạo policy/authorization handler ở service sở hữu resource. Ví dụ policy `CanReadOrder` đọc `sub`, tải order bằng repository của Ordering, rồi xác nhận CustomerId trùng. Không nhét rule này vào Gateway vì Gateway không sở hữu order.

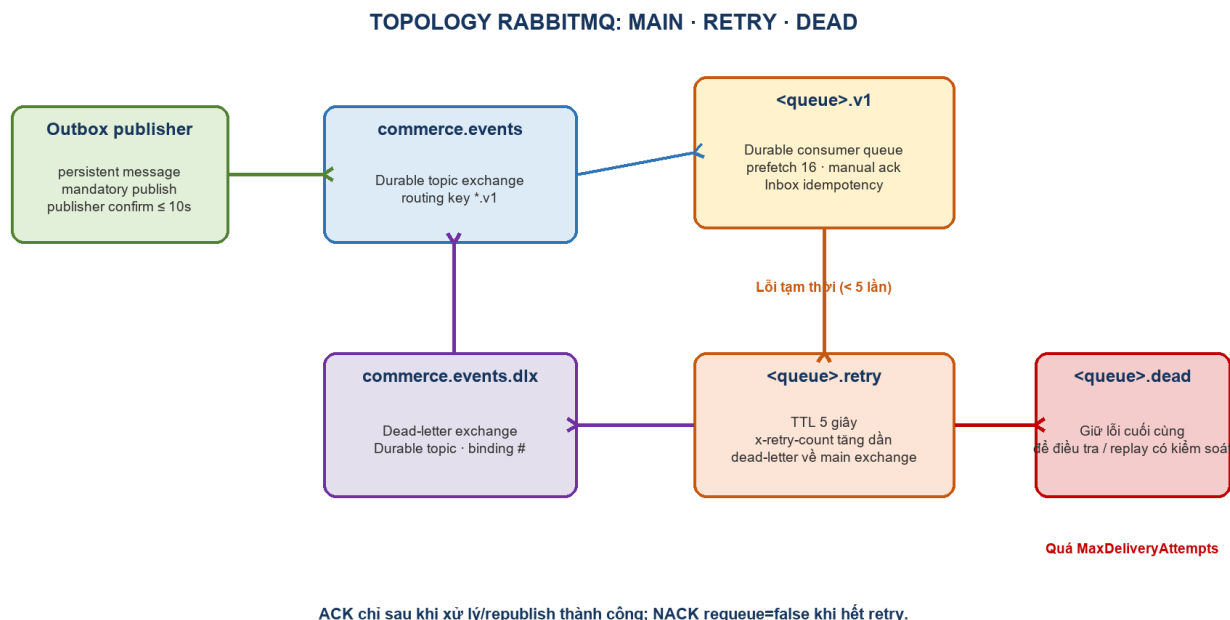
```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CatalogWrite", policy =>
        policy.RequireRole("Admin")
            .RequireClaim("scope", "catalog.write"));
});

// Endpoint vẫn phải kiểm tra ownership nếu resource gắn với customer.
```

6.4 Checklist thêm endpoint

- Quyết định endpoint public, authenticated, role-based hay resource-based.
- Khai báo `[Authorize]`, role/policy tại service; không chỉ dựa vào route Gateway.
- Không nhận CustomerId từ body nếu đã có `sub`; tránh IDOR/mạo danh.
- Trả 401 khi thiếu/sai token, 403 khi đã xác thực nhưng không đủ quyền, 404 khi cần tránh lộ sự tồn tại resource.
- Cập nhật Postman collection khi REST contract/auth workflow thay đổi.
- Thêm test cho anonymous, sai role, đúng role và sai ownership.

7. RabbitMQ: cấu hình, độ tin cậy, retry, DLQ và tùy biến



Hình 4 — Đường đi của message khi thành công, retry và lỗi cuối cùng.

7.1 Cấu hình hiện tại

Khóa	Giá trị/nguồn	Ý nghĩa
RabbitMq_ConnectionString	amqp://<user>:<pass>@rabbitmq:5672	AMQP nội bộ Docker
RabbitMq_ExchangeName	commerce.events	Durable topic exchange chính
RabbitMq_DeadLetterExchangeName	commerce.events.dlx	Durable topic DLX
MaxDeliveryAttempts	5	Số lần thử trước khi dead-letter cuối
Retry delay	5.000 ms	TTL của queue retry
Prefetch	16	Tối đa message chưa ACK trên channel/consumer
Publisher confirm timeout	10 giây	Publisher chỉ hoàn tất khi broker confirm

7.2 Envelope và trace context

Publisher dùng message persistent, `mandatory=true` và publisher confirms. Message mang MessageId, CorrelationId, Type/routing key, JSON content, cùng `traceparent`, `tracestate` và causation id. Schema contract không phải domain entity; chỉ chứa dữ liệu cần để service khác phản ứng.

7.3 Publisher: Outbox

Outbox message được ghi trong cùng local transaction với Order/Reservation/Payment. Worker đọc theo batch 20, claim lease 30 giây, poll khoảng 2 giây khi rỗi và nhanh hơn khi còn việc. Publisher thử tối đa 10 lần; chỉ đánh dấu processed sau khi broker confirm. Vì crash có thể xảy ra sau publish nhưng trước mark-processed, consumer vẫn phải idempotent.

7.4 Consumer: ACK, retry và Inbox

1. Broker giao message, consumer dùng manual ACK và prefetch 16.
2. Consumer mở local transaction, kiểm tra Inbox theo MessageId + Consumer.
3. Nếu chưa xử lý, thực hiện mutation và ghi Inbox trong cùng transaction.
4. Thành công thì ACK. Lỗi tạm thời thì publish sang retry queue, tăng `x-retry-count`, sau đó ACK bản gốc.
5. Retry queue hết TTL 5 giây sẽ dead-letter về exchange chính/routing key ban đầu.
6. Đủ 5 lần vẫn lỗi thì NACK `requeue=false`; broker đưa vào DLX và `

7.5 Routing key và queue quan trọng

Routing key	Queue	Consumer
ordering.inventory-reservation-requested.v1	inventory.reserve-order.v1	Inventory
inventory.reserved.v1	ordering.inventory-reserved.v1	Ordering
inventory.reservation-failed.v1	ordering.inventory-failed.v1	Ordering
ordering.payment-requested.v1	payment.process-order.v1	Payment
payment.succeeded.v1	ordering.payment-succeeded.v1	Ordering
payment.failed.v1	ordering.payment-failed.v1	Ordering
ordering.order-paid.v1	inventory.confirm-paid-order.v1	Inventory
ordering.order-paid.v1	shipping.create-paid-order.v1	Shipping
shipping.shipment-created.v1	ordering.shipment-created.v1	Ordering
.order/payment/shipment events	notification..v1	Notification

7.6 Dùng RabbitMQ Management UI

1. Mở `http://localhost:15672` và đăng nhập bằng `RABBITMQ\_DEFAULT\_USER/PASS` trong `.env`.
2. Overview: xem message rate, connection/channel count và node alarm. Alarm memory/disk có thể làm publisher bị flow control.
3. Exchanges: mở `commerce.events`, xem type=topic, durability và bindings; mở DLX để kiểm tra dead bindings.
4. Queues and Streams: tìm queue consumer; theo dõi Ready, Unacked, Total, Consumers, Incoming và Deliver/Ack.
5. Connections/Channels: xác nhận từng container còn kết nối; Unacked tăng mà ack rate đứng yên thường là consumer treo/chậm.
6. Chỉ dùng Get messages khi debug local. Chọn requeue=true nếu không muốn lấy message khỏi queue.

7.7 Replay dead message an toàn

1. Chụp MessageId, routing key, correlation id, headers và exception/log liên quan.
2. Sửa nguyên nhân gốc trước: schema, dependency, dữ liệu hoặc bug handler.
3. Kiểm tra handler có Inbox/idempotency và side effect ngoài DB (email/payment) có idempotency riêng.
4. Republish vào `commerce.events` bằng routing key gốc, giữ MessageId/correlation nếu tool cho phép; không publish vào queue retry tùy tiện.
5. Theo dõi ACK, Inbox và trạng thái nghiệp vụ. Chỉ sau đó mới purge/delete dead message nếu cần.

7.8 Thêm event hoặc consumer mới

1. Định nghĩa integration contract độc lập domain entity; đặt tên sự kiện ở thì quá khứ nếu là fact, hoặc `\*-requested` nếu là command-like event.
2. Chọn routing key `

3. Đăng ký durable queue riêng của consumer, main binding, retry queue, dead queue và Inbox identity.
4. Publisher ghi Outbox trong cùng transaction. Consumer ACK sau commit; propagate cancellation và trace context.
5. Test duplicate delivery, poison message, broker restart, consumer restart và thứ tự event. Không giả định global ordering giữa nhiều queue.
6. Cập nhật tài liệu/ma trận contract và dashboard backlog/dead queue.

7.9 Hardening production

- Tạo vhost và user riêng cho từng service; cấp configure/write/read đúng exchange/queue cần thiết, không dùng shared admin account.
- Bật TLS, quản lý certificate/credential bằng secret manager, xoay credential và không publish AMQP port công khai.
- Đặt queue length/TTL/policy theo tải, quorum queues nếu cần HA, cảnh báo dead queue và consumer lag.
- Chỉ replay tự động khi có invariant rõ; với payment/email nên ưu tiên quy trình phê duyệt.

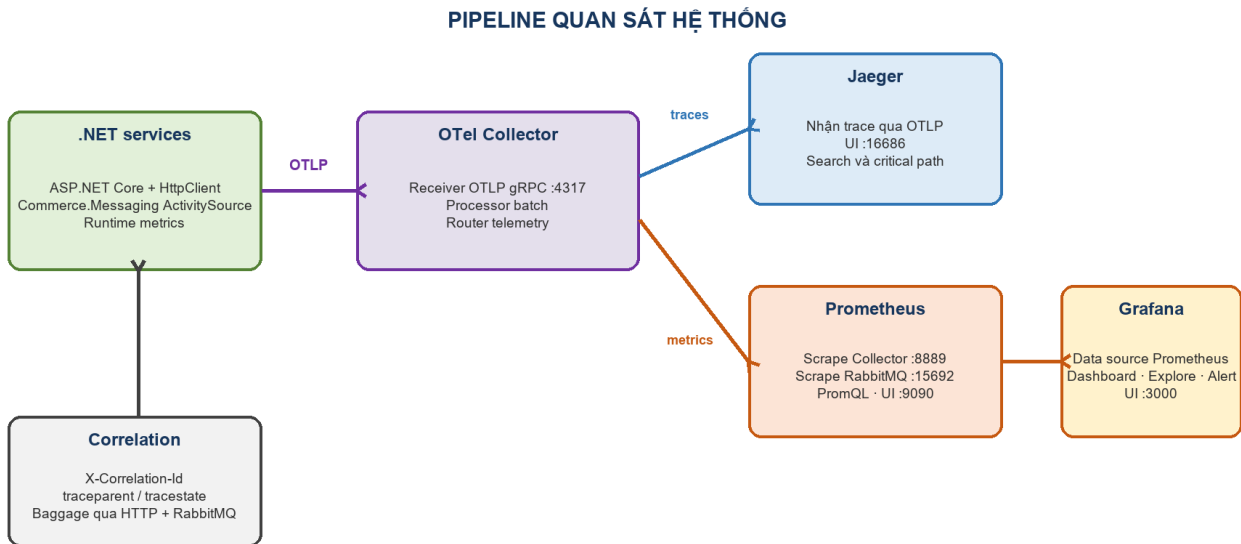
7.10 Kiểm thử failure có chủ đích

1. Dừng consumer sau khi publisher đã ghi Outbox; khởi động lại và xác nhận message vẫn được xử lý.
2. Gây lỗi handler tạm thời; xác nhận message đi qua retry queue, `x-retry-count` tăng và không tạo mutation trùng.
3. Gây poison message; xác nhận message vào `.dead` sau giới hạn retry và queue chính tiếp tục phục vụ message khác.
4. Restart RabbitMQ; xác nhận durable exchange/queue và persistent message còn tồn tại, publisher tự kết nối lại.
5. Gửi lại cùng MessageId; xác nhận Inbox bỏ qua side effect lần hai.

7.11 Chỉ số nên cảnh báo

- Ready tăng liên tục: consumer không theo kịp hoặc đã mất kết nối.
- Unacked cao kéo dài: handler chậm/treo, prefetch quá lớn hoặc downstream bị nghẽn.
- Consumers = 0 trên queue nghiệp vụ: container/registration gặp lỗi.
- Dead queue > 0: cần ticket/điều tra ngay; không chỉ purge để làm biểu đồ xanh.

8. OpenTelemetry Collector: thu nhận và định tuyến telemetry



Grafana không nhận telemetry trực tiếp; nó truy vấn Prometheus để hiển thị và cảnh báo.

Hình 5 — Trace và metric dùng chung OTLP receiver nhưng đi tới backend khác nhau.

8.1 Collector dùng để làm gì

OpenTelemetry Collector là agent/gateway trung lập vendor. Service gửi telemetry một lần bằng OTLP; Collector nhận, xử lý theo batch và xuất tới backend. Collector không có dashboard người dùng cuối. Trong stack hiện tại, trace đi tới Jaeger; metric được chuyển thành endpoint Prometheus scrape.

8.2 Pipeline hiện tại

```

receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
processors:
  batch: {}
exporters:
  otlp_grpc/jaeger:
    endpoint: jaeger:4317
    tls: { insecure: true }
  prometheus:
    endpoint: 0.0.0.0:8889
service:
  pipelines:
    traces: { receivers: [otlp], processors: [batch], exporters: [otlp_grpc/jaeger] }
    metrics: { receivers: [otlp], processors: [batch], exporters: [prometheus] }
  
```

Mỗi .NET service đặt `service.name` bằng application name, instrument ASP.NET Core/HttpClient/runtime và ActivitySource `Commerce.Messaging`. Docker cung cấp `OTEL\_EXPORTER\_OTLP\_ENDPOINT=http://otel-collector:4317`, protocol gRPC và resource attribute `deployment.environment=local`.

8.3 Kiểm tra Collector

```
docker compose --env-file .env ps otel-collector jaeger prometheus
docker compose --env-file .env logs --tail 200 otel-collector

# Từ host, Prometheus phải thấy target commerce-otel là UP:
Invoke-RestMethod http://localhost:9090/api/v1/targets
```

- `connection refused` tới jaeger:4317: kiểm tra network/service name và Jaeger readiness.
- Metric target down: kiểm tra Collector còn listen 8889 và Prometheus cùng network.
- Có HTTP request nhưng không có trace: kiểm tra OTLP endpoint/protocol, sampling và log Collector.

8.4 Tùy biến an toàn

Khi tải tăng, thêm `memory\_limiter` trước `batch`; khi cần chuẩn hóa metadata, thêm `resource`/`attributes` processor; khi điều tra local, có thể thêm debug exporter tạm thời. Mỗi processor chỉ có hiệu lực khi được đưa vào đúng pipeline.

```
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 256
  batch:
    timeout: 5s
    send_batch_size: 1024
  attributes/redact:
    actions:
      - key: http.request.header.authorization
        action: delete

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes/redact, batch]
      exporters: [otlp_grpc/jaeger]
```

Sau khi sửa: Chạy `docker compose config`, restart Collector, đọc log lỗi cấu hình, tạo request thử, kiểm tra Jaeger/Prometheus. Không gửi PII/high-cardinality ID vào metric label.

8.5 Sampling

Local nên giữ 100% trace để học/debug. Production có thể dùng parent-based ratio sampling tại SDK để giảm dữ liệu, hoặc tail sampling tại Collector nếu cần giữ toàn bộ trace lỗi/chậm. Tail sampling cần Collector đủ memory và kiến trúc phân phối phù hợp; không bật chỉ bằng một dòng mà không capacity test.

9. Prometheus: scrape, PromQL, recording rule và alert rule

9.1 Prometheus dùng để làm gì

Prometheus kéo metric theo chu kỳ, lưu chuỗi thời gian và truy vấn bằng PromQL. Cấu hình hiện tại scrape mỗi 10 giây từ Collector `otel-collector:8889` và RabbitMQ `rabbitmq:15692`. Do exporter cần tránh xung đột label `job`, `service.name` xuất hiện dưới label `exported\_job` trong các truy vấn dashboard hiện tại.

9.2 Quy trình sử dụng UI

1. Mở `http://localhost:9090/targets`. Hai scrape pool `commerce-otel` và `rabbitmq` phải UP.
2. Mở Graph/Query; bắt đầu với `up`, chuyển Table để xem labels trước khi dựng biểu đồ.
3. Thu hẹp theo `exported\_job`, `http\_route` hoặc label RabbitMQ; tránh query không giới hạn trên khoảng thời gian dài.
4. Dùng Range query và chọn time window; so sánh rate thay vì raw counter.
5. Nếu query đúng, chuyển sang Grafana để tạo panel lâu dài; Prometheus UI phù hợp điều tra nhanh.

9.3 PromQL thực dụng

Mục tiêu	PromQL
Target health	up
Request/giây theo service	sum by (exported_job) (rate(http_server_request_duration_seconds_count[5m]))
P95 latency theo service	histogram_quantile(0.95, sum by (le, exported_job) (rate(http_server_request_duration_seconds_bucket[5m])))
P95 theo route	histogram_quantile(0.95, sum by (le, exported_job, http_route) (rate(http_server_request_duration_seconds_bucket[5m])))
RabbitMQ ready	sum by (queue) (rabbitmq_queue_messages_ready)
RabbitMQ unacked	sum by (queue) (rabbitmq_queue_messages_unacked)
Consumer count	sum by (queue) (rabbitmq_queue_consumers)

Cardinality: Không đưa customer id, order id, email, URL thô hoặc correlation id thành metric label. Dùng trace/log cho dữ liệu từng request; metric chỉ nên có chiều hữu hạn như service, route chuẩn hóa, method, status class.

9.4 Thêm scrape target

```
scrape_configs:
  - job_name: commerce-otel
    static_configs:
      - targets: [otel-collector:8889]
  - job_name: rabbitmq
    static_configs:
      - targets: [rabbitmq:15692]
  - job_name: custom-exporter
    scrape_interval: 15s
    static_configs:
      - targets: [custom-exporter:9100]
```

Target phải truy cập được từ container Prometheus. `localhost` trong container là chính Prometheus, không phải Windows và cũng không phải service khác.

9.5 Recording rule và alert rule

```
groups:
- name: commerce-http
  interval: 30s
  rules:
    - record: commerce:http_requests:rate5m
      expr: sum by (exported_job) (rate(http_server_request_duration_seconds_count[5m]))
    - alert: CommerceTargetDown
      expr: up{job=~"commerce-otel|rabbitmq"} == 0
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "Commerce scrape target is down"
```

Để dùng rule, mount file vào container và thêm `rule\_files` trong `deploy/prometheus.yaml`. Alerting rule chỉ tạo alert state; muốn gửi thông báo theo Prometheus-native cần Alertmanager, hiện chưa có trong Compose. Có thể dùng Grafana Alerting thay thế cho local/demo.

9.6 Kiểm tra sau khi custom

```
docker compose --env-file .env config
docker compose --env-file .env exec -T prometheus `
  promtool check config /etc/prometheus/prometheus.yml
docker compose --env-file .env restart prometheus
```

9.7 Khi query trả về rỗng

1. Chạy `up` để xác nhận target còn được scrape; mở `/targets` đọc Last Scrape Error.
2. Query tên metric không có filter, chuyển sang Table và xem label thực tế trước khi viết selector.
3. Tạo traffic trong time range đang xem; counter/rate sẽ không có dữ liệu hữu ích nếu chưa có request.
4. Đối với `rate`, chọn range vector đủ lớn so với scrape interval 10 giây, thường từ 1–5 phút.
5. Kiểm tra metric có bị đổi tên/đơn vị bởi OTLP-to-Prometheus exporter sau khi nâng version.

9.8 Quy tắc alert tốt

- Alert theo tác động người dùng hoặc nguy cơ mất dữ liệu, không chỉ theo một spike kỹ thuật ngắn.
- Dùng `for` để giảm nhiễu; labels phục vụ routing, annotations giải thích tác động và runbook.
- Test trạng thái Pending/Firing/Resolved và trường hợp Prometheus thiếu dữ liệu.
- Recording rule nên có tên ổn định, đơn vị rõ và giúp query dashboard/alert dễ đọc hơn.

10. Grafana: dashboard, Explore, biến, cảnh báo và provisioning

10.1 Grafana dùng để làm gì

Grafana là lớp hiển thị và alerting; nó không lưu metric ứng dụng thay Prometheus. Stack provision sẵn data source Prometheus (`http://prometheus:9090`) và dashboard `Commerce overview`, gồm request rate và p95 latency. Tài khoản admin lấy từ `GRAFANA\_ADMIN\_USER/PASSWORD` trong `.env`.

10.2 Xem dashboard và điều tra bằng Explore

1. Mở `http://localhost:3000`, đăng nhập, vào Dashboards → Commerce → Commerce overview.
2. Chọn time range đủ bao phủ lúc test; bật Auto refresh 5s hoặc 10s khi chạy smoke test.
3. Panel request rate cho biết lưu lượng; p95 cho biết 95% request nhanh hơn ngưỡng hiển thị.
4. Chọn Explore → Prometheus, chạy `up`, sau đó paste PromQL từ Chương 9.
5. Dùng Inspect → Query/Data để phân biệt “không có dữ liệu” với lỗi query hoặc lỗi datasource.

10.3 Tạo panel mới

1. Tạo dashboard mới hoặc copy dashboard provisioned sang dashboard editable.
2. Add visualization, chọn Prometheus, nhập query p95 theo route.
3. Legend dùng `{{exported\_job}} · {{http\_route}}`; Unit chọn seconds; Min=0.
4. Thêm threshold theo SLO nội bộ, ví dụ warning 0,5s và critical 1s; đây là ngưỡng minh họa, phải hiệu chỉnh từ yêu cầu thật.
5. Đổi time range và tạo traffic để xác nhận series/legend. Lưu dashboard với tên/thu mục rõ ràng.

10.4 Tạo biến service

```
Variable name: service
Type: Query
Data source: Prometheus
Query: label_values(http_server_request_duration_seconds_count, exported_job)

# Sau đó lọc panel:
sum by (http_route) (
  rate(http_server_request_duration_seconds_count{exported_job=~"$service"}[5m])
)
```

Bật Multi-value/Include All nếu cần so sánh nhiều service; dùng toán tử regex `=~` thay vì `=` cho multi-value.

10.5 Provisioning và lưu thay đổi

Repo mount `deploy/grafana/provisioning` và `deploy/grafana/dashboards` ở chế độ read-only. Provider scan mỗi 30 giây. `allowUiUpdates` không được bật, vì vậy cách bền vững là chỉnh dashboard JSON trong repo hoặc export dashboard mới thành JSON rồi review/commit. Không chỉ sửa trong UI và kỳ vọng thay đổi tồn tại sau recreate container.

```
# Sau khi chỉnh provisioning hoặc JSON dashboard
docker compose --env-file .env restart grafana
docker compose --env-file .env logs --tail 100 grafana
```

10.6 Tạo alert trong Grafana

1. Alerting → Alert rules → New alert rule; chọn Prometheus query, ví dụ target down hoặc p95 vượt ngưỡng.
2. Thêm Reduce/Threshold expression và `for` duration để tránh alert do spike ngắn.

3. Gắn labels như team, service, severity; annotations mô tả tác động và runbook URL.
4. Tạo Contact point và Notification policy. Stack local hiện chưa provision kênh gửi thông báo.
5. Test bằng cách dừng một service có chủ đích, xác nhận Pending → Firing → Resolved, rồi khởi động lại.

10.7 Dashboard nên bổ sung

- HTTP: request rate, error ratio, p50/p95/p99 theo service/route.
- RabbitMQ: ready, unacked, consumer count, publish/deliver/ack rate và dead queue.
- Runtime: process CPU/memory, GC pause/heap, thread pool queue nếu exporter cung cấp.
- Saga: cần thêm business metrics trong code như checkout started/completed/cancelled và duration; không suy diễn hoàn toàn từ HTTP.

10.8 Export và review dashboard

1. Sau khi thử nghiệm trong UI, export dashboard JSON và bỏ datasource id phụ thuộc instance nếu cần portability.
2. Đặt `uid` ổn định, title/folder rõ ràng; review PromQL, unit, legend, threshold và time range mặc định.
3. Lưu JSON dưới `deploy/grafana/dashboards`, restart Grafana và xác nhận provisioner nạp bản mới.
4. Không commit credential, contact-point secret hoặc URL chứa token trong dashboard JSON.

10.9 Bố cục dashboard vận hành

- Hàng 1 — Tình trạng: target up, request rate, error ratio, p95/p99.
- Hàng 2 — Dependency: RabbitMQ backlog/unacked/consumer, database/Redis health nếu có exporter.
- Hàng 3 — Runtime: CPU, memory, GC, thread pool; dùng cùng biến service/environment.
- Panel phải có mô tả ngắn và link sang runbook/Jaeger để operator biết bước tiếp theo.

11. Jaeger: tìm trace, đọc critical path và tùy biến

11.1 Jaeger dùng để làm gì

Jaeger lưu và hiển thị distributed trace. Một trace gồm các span HTTP, gRPC và RabbitMQ producer/consumer liên quan. Nó giúp trả lời request chậm ở đâu, lỗi nằm ở hop nào, retry có xảy ra và context có truyền qua event hay không. Image local hiện là Jaeger 2.21.0, all-in-one/in-memory; restart có thể làm mất trace.

11.2 Tìm và đọc trace

1. Tạo traffic trước; mở `http://localhost:16686`.
2. Chọn service `Commerce.Gateway`, `Ordering.Api` hoặc service liên quan; chọn Lookback bao phủ thời điểm gửi request.
3. Lọc Operation nếu danh sách dài; có thể dùng tags/status khi telemetry đã gắn thuộc tính.
4. Mở trace, đọc từ root span; thanh dài nhất trên đường phụ thuộc thường là critical path.
5. Mở span để xem duration, status, resource/service, HTTP/gRPC attributes và event/exception. Không kỳ vọng thấy body/token vì không nên thu thập.
6. Với checkout, tìm producer span `Commerce.Messaging`, rồi consumer span ở Inventory/Payment/Shipping. Async gap là bình thường.

11.3 Khi không thấy trace

Triệu chứng	Kiểm tra	Cách xử lý
Không có service	Log app và Collector	Kiểm tra OTLP endpoint/protocol, network, sampling
Có HTTP nhưng mất RabbitMQ	Message headers + consumer log	Kiểm tra traceparent/tracestate propagation và ActivitySource
Trace rời rạc	Parent/span context	Đảm bảo consumer khởi tạo Activity từ extracted context
Service trùng/khó hiểu	resource.service.name	Đặt service name ổn định, thêm environment/instance attributes
Trace biến mất sau restart	Storage backend	Đây là giới hạn in-memory local; dùng storage bền vững ở production

11.4 Tùy biến instrumentation

Tạo custom span cho nghiệp vụ dài bằng ActivitySource đã đăng ký. Tên span nên ổn định và mô tả thao tác, ví dụ `checkout.validate-cart`, không chứa order id trong tên. Order id có thể là span attribute nếu chính sách dữ liệu cho phép, nhưng không dùng làm metric label. Gắn status Error và record exception có chọn lọc; không ghi credential/body nhạy cảm.

```
using var activity = CommerceTelemetry.ActivitySource.StartActivity(
    "checkout.validate-cart",
    ActivityKind.Internal);

activity?.SetTag("commerce.operation", "checkout");
// Không gắn token, password hoặc dữ liệu cá nhân vào span.
```

11.5 Tùy biến UI/storage

- Local: có thể cấu hình UI bằng Jaeger UI config, nhưng giữ Compose đơn giản nếu không có nhu cầu cụ thể.
- Production: dùng storage bền vững được hỗ trợ, thiết lập retention/index, capacity và bảo vệ UI bằng SSO/reverse proxy.

- Không mở cổng ingest/UI công khai. Bật TLS và xác thực giữa Collector với backend.

11.6 Đọc trace bất đồng bộ qua RabbitMQ

1. Bắt đầu từ HTTP root span và ghi lại trace id, correlation id cùng thời điểm checkout.
2. Tìm producer span của `Commerce.Messaging`; kiểm tra routing key và publish duration/status.
3. Đi theo consumer span ở service kế tiếp. Khoảng trống giữa producer và consumer là thời gian nằm trong broker/retry, không nhất thiết là mất trace.
4. Nếu consumer span có parent/link sai, kiểm tra bước extract `traceparent`/`tracestate` trước khi tạo Activity.
5. Đối chiếu span lỗi với Ready/Unacked/dead queue và log cùng correlation id; Jaeger không thay thế trạng thái broker.

11.7 Checklist khi đưa Jaeger lên production

- Chọn storage và retention theo tốc độ ingest; kiểm thử truy vấn khi index lớn.
- Đặt sampling policy có thể giải thích; ưu tiên giữ trace lỗi/chậm nhưng không vượt ngân sách.
- Bảo vệ Query UI/API bằng SSO/RBAC và network policy; mã hóa đường Collector → Jaeger.
- Cảnh báo Collector export failure và storage saturation; backup cấu hình, không coi trace là system of record.

12. Runbook xử lý sự cố và hardening production

12.1 Runbook: API trả 401

1. Xác nhận header đúng dạng `Authorization: Bearer <token>` và token chưa hết hạn.
2. So issuer/audience ở `.env` với cấu hình tất cả container; thay `.env` cần recreate container để nhận biến mới.
3. Kiểm tra clock host/container; clock skew hiện 30 giây.
4. Đăng nhập lại để loại trừ token cũ. Không in token vào log khi debug.
5. Đọc log Gateway và service đích; cả hai đều validate token.

12.2 Runbook: API trả 403

1. Decode claim offline để xác nhận `role` đúng `Admin`/`Customer`; role claim type hiện là `role`.
2. Phân biệt role failure với ownership failure. Token hợp lệ nhưng resource không thuộc user vẫn phải bị từ chối.
3. Kiểm tra user seed và role assignment trong Identity DB; không sửa token thủ công.

12.3 Runbook: checkout đứng ở trạng thái trung gian

1. Lấy order id/correlation id; tìm trace ở Jaeger.
2. RabbitMQ UI: kiểm tra queue tương ứng Ready, Unacked, Consumers và `.dead`.
3. Log Outbox publisher/consumer service; xác nhận connection và confirm.
4. Kiểm tra Outbox/Inbox chỉ trong database service liên quan; không sửa trực tiếp trạng thái saga.
5. Nếu dead message: sửa nguyên nhân, xác minh idempotency rồi replay có kiểm soát.

12.4 Runbook: dashboard trống

1. Tạo request thực tế và chọn đúng time range.
2. Prometheus `/targets`: commerce-otel/rabbitmq phải UP.
3. Prometheus query `up`, rồi query metric gốc; xem labels thực tế trước khi filter.
4. Grafana data source Save & Test; URL phải là `http://prometheus:9090` trong Docker, không phải localhost.
5. Collector/app logs: tìm lỗi OTLP export hoặc config pipeline.

12.5 Runbook: Navicat/Redis client không kết nối

Tool	Thiết lập	Lỗi thường gặp
PostgreSQL client	Host 127.0.0.1; port POSTGRES_PORT; user từ .env; database cụ thể như commerce_identity	Đề Initial Database trống có thể client thử database cùng tên user và báo database không tồn tại
Redis client	Host 127.0.0.1; port REDIS_PORT hoặc 6379; Authentication=None	Container chưa recreate sau khi thêm ports; port host bị chiếm; mapping chỉ bind loopback

```
docker compose --env-file .env ps postgres redis
docker compose --env-file .env port postgres 5432
docker compose --env-file .env port redis 6379
Test-NetConnection 127.0.0.1 -Port 5432
Test-NetConnection 127.0.0.1 -Port 6379
```

12.6 Production hardening theo lớp

Lớp	Việc cần làm
Identity/JWT	OIDC + asymmetric signing/JWKS; key rotation; secret manager; MFA/admin policy
Authorization	Policy + ownership checks; scope/permission rõ; audit security decisions
Service-to-service	mTLS/workload identity; network policy; authenticate gRPC
RabbitMQ	TLS; per-service user/vhost/permission; HA/quorum; alert DLQ/backlog
Database/Redis	Không publish public; TLS/auth; least privilege; backup/restore test
Observability	SSO/RBAC; TLS; retention; redaction; sampling; cardinality budget
Resilience	Timeout, retry có jitter, circuit breaker ở call sync; capacity/load test
Delivery	Không commit `.env`; image pinning/SBOM/scan; migration và rollback có kiểm soát

12.7 Definition of done cho thay đổi kiến trúc

- Build/test qua; không phá Clean Architecture dependency direction.
- Public REST contract, role/ownership và Postman collection đồng bộ.
- gRPC `.proto` backward-compatible hoặc được version rõ.
- Integration event, routing key, queue, retry/DLQ, Outbox/Inbox được cập nhật và test.
- Metric/trace không chứa secret/PII và không tăng cardinality vô hạn.
- Dashboard/alert/runbook/documentation cập nhật cùng code.

Phụ lục A. Lệnh vận hành nhanh

A.1 Docker Compose

```
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
docker compose --env-file .env logs -f --tail 200 ordering-api inventory-api payment-api
docker compose --env-file .env restart otel-collector prometheus grafana
docker compose --env-file .env config
```

A.2 Health và observability

```
Invoke-WebRequest -UseBasicParsing http://localhost:8080/health/ready
Invoke-RestMethod http://localhost:9090/api/v1/targets
Start-Process http://localhost:15672 # RabbitMQ
Start-Process http://localhost:16686 # Jaeger
Start-Process http://localhost:9090 # Prometheus
Start-Process http://localhost:3000 # Grafana
```

A.3 Database local

```
docker compose --env-file .env exec -T postgres `
    psql -U <POSTGRES_USER> -d commerce_ordering -c '\dt'

docker compose --env-file .env exec -T redis redis-cli ping
```

Phụ lục B. Danh mục routing key đầy đủ

Routing key	Publisher	Consumer(s)
ordering.order-created.v1	Ordering	Notification
ordering.inventory-reservation-requested.v1	Ordering	Inventory
ordering.inventory-release-requested.v1	Ordering	Inventory
ordering.payment-requested.v1	Ordering	Payment
ordering.order-paid.v1	Ordering	Inventory, Shipping
ordering.order-cancelled.v1	Ordering	Notification
inventory.reserved.v1	Inventory	Ordering
inventory.reservation-failed.v1	Inventory	Ordering
inventory.released.v1	Inventory	Ordering
inventory.reservation-expired.v1	Inventory	Ordering
payment.succeeded.v1	Payment	Ordering, Notification
payment.failed.v1	Payment	Ordering, Notification
shipping.shipment-created.v1	Shipping	Ordering, Notification
shipping.shipment-delivered.v1	Shipping	Ordering, Notification

B.1 Các queue consumer

- inventory.reserve-order.v1 · inventory.release-order.v1 · inventory.confirm-paid-order.v1
- payment.process-order.v1
- shipping.create-paid-order.v1
- ordering.inventory-reserved.v1 · ordering.inventory-failed.v1 · ordering.inventory-expired.v1 · ordering.inventory-released.v1
- ordering.payment-succeeded.v1 · ordering.payment-failed.v1 · ordering.shipment-created.v1 · ordering.shipment-delivered.v1
- notification.order-created.v1 · notification.order-cancelled.v1 · notification.payment-succeeded.v1 · notification.payment-failed.v1
- notification.shipment-created.v1 · notification.shipment-delivered.v1

Mỗi main queue có thêm ``<queue>.retry`` và ``<queue>.dead`` theo topology ở Chương 7.

Phụ lục C. Tập cấu hình và mã nguồn cần đọc

Chủ đề	Vị trí
Docker topology/secret mapping	docker-compose.yml; .env.example
Gateway/YARP/rate limit	src/Gateway/Commerce.Gateway/appsettings.json; Program.cs
JWT validation	src/BuildingBlocks/Commerce.BuildingBlocks.Infrastructure/Security
RabbitMQ/Outbox/Inbox	src/BuildingBlocks/Commerce.BuildingBlocks.Infrastructure/Messaging
Integration contracts	src/BuildingBlocks/Commerce.BuildingBlocks.Contracts/IntegrationEvents
OTel Collector	deploy/otel-collector.yaml
Prometheus	deploy/prometheus.yaml
Grafana provisioning	deploy/grafana/provisioning; deploy/grafana/dashboards
Kiến trúc hiện có	docs/architecture.md; docs/local-infrastructure-guide.md

Phụ lục D. Tài liệu tham khảo chính thức

1. [ASP.NET Core JWT bearer authentication](#)
2. [RabbitMQ Consumer Acknowledgements and Publisher Confirms](#)
3. [RabbitMQ Exchanges](#)
4. [RabbitMQ Management Plugin](#)
5. [OpenTelemetry Collector](#)
6. [OpenTelemetry Collector Components](#)
7. [Prometheus Querying Basics](#)
8. [Prometheus Recording Rules](#)
9. [Prometheus Alerting Rules](#)
10. [Grafana Dashboards](#)
11. [Grafana Dashboard Variables](#)
12. [Grafana Provisioning](#)
13. [Jaeger 2.21 Documentation](#)
14. [Jaeger Storage](#)
15. [Jaeger Troubleshooting](#)

Kết luận

Commerce tách rõ giao tiếp công khai, truy vấn nội bộ và choreography bất đồng bộ. Độ tin cậy không đến từ một công cụ đơn lẻ mà từ tổ hợp timeout, idempotency, Outbox/Inbox, publisher confirm, retry/DLQ và quan sát xuyên suốt. Khi custom, hãy thay đổi nhỏ, validate cấu hình, tạo traffic kiểm chứng, rồi đối chiếu cả metric, trace, queue và trạng thái domain.